

Jaxer Guide

- Introduction
- Setup
- Develop
 - Application Level Concepts
 - Persistent Storage
 - Communications
 - Utilities
 - Advanced Topics
 - Using `__noSuchMethod__`
 - Writing Jaxer Extensions
- Debug

Advanced Topics in Jaxer

`__noSuchMethod__`

By default, an attempt to call a method that doesn't exist on an object results in a `TypeError` being thrown. This behavior can be avoided by defining a function at that object's `__noSuchMethod__` member. The function attached will be invoked with two arguments, the name of the method called and an array of the arguments passed in the method call.

The [default Mozilla implementation](#) omits allowing this to happen on the anonymous global (window) object so while a call to the method in the window namespace (e.g., `window.foo()`) would be trapped by the `__noSuchMethod__` handler the same function accessed without the window prefix (e.g., `foo()`) would not. Since both calls are referring to the same global object this is an unintuitive result.

Jaxer *repairs* this core JavaScript behavior by making the anonymous global also trigger the `__noSuchMethod__` handler if defined. This was used, to give a real world example, to develop the autoloading component of the [ActiveJS framework](#).

To demonstrate the difference between the old and new handling of missing methods, the sample below shows different results when the handler is triggered by server code (new behavior) versus client code (old behavior). The second method call on the client results in an exception, whereas both calls are trapped by the handler on the server side. The try/catch blocks avoid messy browser exceptions.

Note: The following sample will only work in Firefox as it illustrates in the feature between the Jaxer and Mozilla engines.

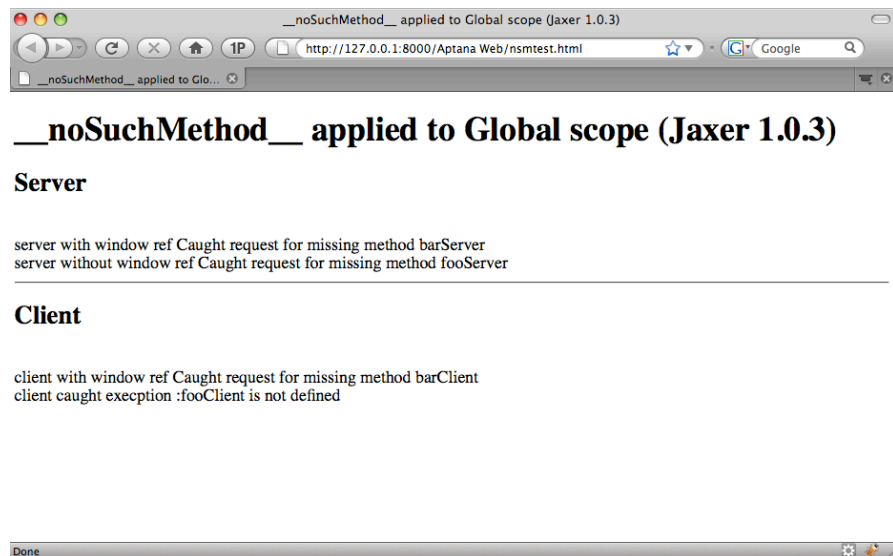
Contents

- [__noSuchMethod__](#)
- [Jaxer Extensions](#)

Using the global `__noSuchMethod__` handler

```
01. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
02. <!-- "http://www.w3.org/TR/html4/loose.dtd" -->
03. <html>
04.   <head>
05.     <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
06.     <title>__noSuchMethod__ applied to Global scope (Jaxer 1.0.3)</title>
07.   </head>
08.   <body>
09.     <h1>__noSuchMethod__ applied to Global scope (Jaxer 1.0.3)</h1>
10.
11.     <script runat="both">
12.       __noSuchMethod__ = function noSuchMethod(methodName,args) {
13.         return "Caught request for missing method "+ methodName;
14.       }
15.     </script>
16.
17.     <h2>Server</h2>
18.
19.     <script runat="server">
20.       try
21.       {
22.         document.write('<BR>server with window ref '+window.barServer());
23.         document.write('<BR>server without window ref '+fooServer());
24.       }
25.       catch (e)
26.       {
27.         document.write("<BR>server caught exception"+e.message);
28.       }
29.     </script>
30.
31.     <hr>
32.
33.     <h2>Client</h2>
34.
35.     <script>
36.       try
37.       {
38.         document.write('<BR>client with window ref '+window.barClient());
39.         document.write('<BR>client without window ref '+fooClient());
40.       }
41.       catch (e)
42.       {
43.         document.write("<BR>client caught execption :"+e.message);
44.       }
45.     </script>
46.   </body>
47. </html>
```

Result of running the `__noSuchMethod__` example



Jaxer Extensions

Jaxer is built with a simple method for adding your own capabilities to the core Jaxer object, extensions. An extension is written in JavaScript, in a file stored in the `local_jaxer/extensions` folder, which gets loaded when Jaxer starts up. The functionality can be anything you like but should be structured using the Module pattern developed by Douglas Crockford, which is well explained in a [post on the Yahoo! User Interface blog by Eric Miraglia](#). The pattern basically is a method for keeping all information contained within the module's scope.

The simplest extension would look like this:

```
1. (function(){
2.   Jaxer.MyProp = 'hey I extended Jaxer !!!';
3. })();
```

Jaxer ships with a sample extension, `jaxer/framework/extensions/sampleExtension.js`, which will record the scripts loaded by your applications. As shipped the extension is disabled by having the first line return, so to see it in action you need to comment out that line, restart Jaxer and then run through your application.

Looking at the sample extension's code, in the next block, note that the key work is done in the `onLoad` and `onUnload` event handlers. The two functions add and remove, respectively, the actual functions which the extension use to do the heavy lifting by calling `CoreEvents.addHandler()` and `CoreEvents.removeHandler()`.

Record page and script requests with a Jaxer extension

```
01. (function(){
02.
03.   var PageRecorder = {};
04.   var path = Jaxer.Dir.resolve("resource:///framework/extensions/sample.txt");
05.
06.   function record(text)
07.   {
08.     Jaxer.File.appendLine(path, text);
09.   }
10.
11.   function recordPage(evt)
12.   {
13.     record(Jaxer.request.uri);
14.   }
15.
16.   function recordScript(evt, doc, elt)
17.   {
18.     if (elt.tagName == "SCRIPT")
19.     {
20.       record("src='" + elt.getAttribute("src") + "', runat='" + elt.getAttribute("runat") +
21.         "<br>";
22.     }
23.   }
24.   PageRecorder.onLoad = function onLoad()
25.   {
26.     // Optional - put any kind of initialization here
27.     CoreEvents.addHandler(CoreEvents.PARSE_START, recordPage);
28.     CoreEvents.addHandler(CoreEvents.NEW_ELEMENT, recordScript);
29.   }
30.
31.   PageRecorder.onUnload = function onUnload()
32.   {
33.     // Optional - put any kind of cleanup here
34.     CoreEvents.removeHandler(CoreEvents.NEW_ELEMENT, recordScript);
35.     CoreEvents.removeHandler(CoreEvents.PARSE_START, recordPage);
36.   }
37.
38.   Jaxer.loadExtension("PageRecorder", PageRecorder);
39.
40. })();
```